

Algorithms & Data Structures

Week 01

Ahmet Feyzi Ateş

General Information

- A two phase bootcamp to integrate skills needed to think algorithmically in order to solve problems by using proper data structures
- First phase (5 weeks): Learn;
 - how to analyze a problem, break down into individual steps that can be implemented by a programming language.
 - Evaluate the time and memory costs of your implementation.
- Second phase (5 weeks): Learn;
 - How data is stored in memory
 - How different data structures make a difference while solving a problem.

General Information (1st Phase)

- Data, memory, and CPU relationship
- Algorithmic thinking and problem-solving approach
- How search and sort algorithms work
- Time and memory complexity of algorithms
- Solution analysis with **Big-O** notation
- Evaluating and improving code performance
- Stack, Queue, and Hash Table logic
- Recursion and recursive thinking
- Introduction to Dynamic Programming
- ***Practical examples from different programming languages***

General Information (2nd Phase)

- Data, Memory, and CPU Relationship (revisited)
- Asymptotic Analysis and Big-O (revisited)
- Arrays and Dynamic Arrays
- Single, Double, and Circular Linked Lists
- Stacks and Queues
- Priority Queues
- Hashing, Hash Tables, and Hash Maps
- Binary Trees and Binary Search Trees
- Minimum Heap and Maximum Heap
- Introduction to AVL and Red-Black Trees
- ***Choosing the Right Data Structure in Real-World Problems***

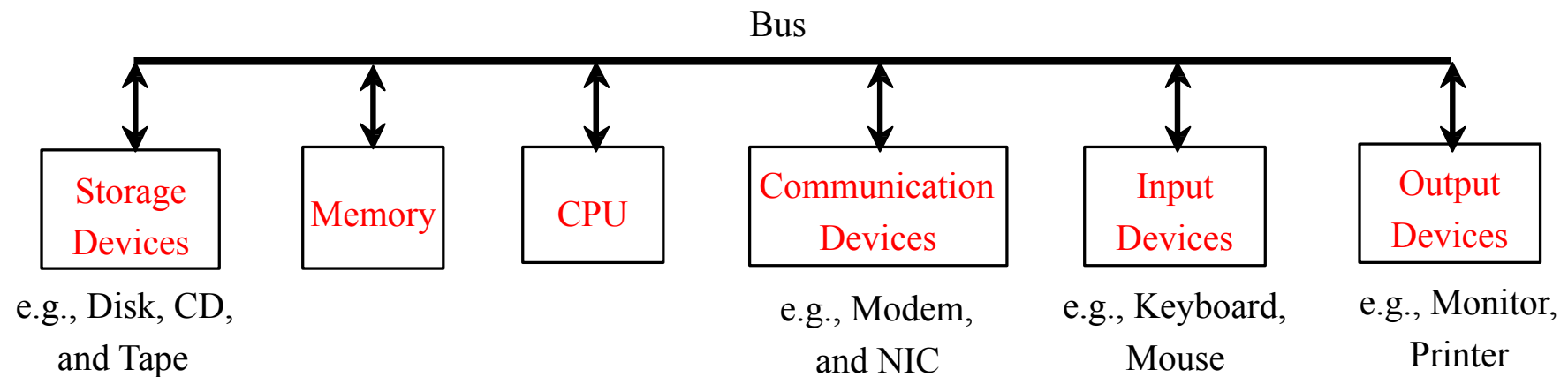
Textbook

Common-Sense Guide to Data Structures and Algorithms, *Level Up Your Core Programming Skills*,
Second Edition, 2020, Jay Wengrow

<https://www.amazon.com.tr/Common-Sense-Guide-Structures-Algorithms-Second/dp/1680507222>

What is a Computer?

A computer consists of a CPU, memory, hard disk, floppy disk, monitor, printer, and communication devices.

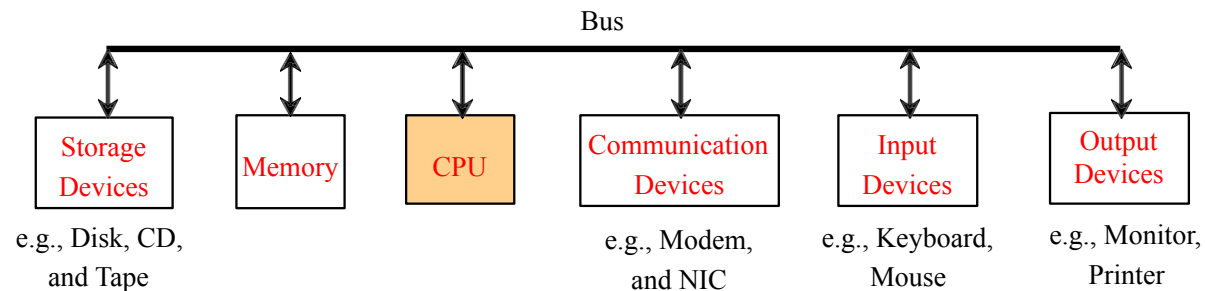


CPU (Central Processing Unit)

The central processing unit (CPU) is the brain of a computer. It retrieves instructions from memory and executes them.

The CPU speed is measured in megahertz (MHz), with 1 megahertz equaling 1 million pulses per second.

The speed of the CPU has been improved continuously.



Memory

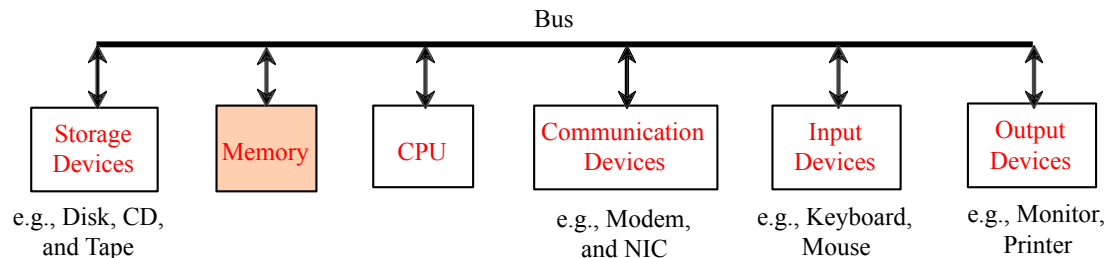
Memory is to store data and program instructions for CPU to execute.

A memory unit is an ordered sequence of bytes, each holds eight bits.

A program and its data must be brought to memory before they can be executed (Stored Program Computer).

A memory byte is never empty, but its initial content may be meaningless to your program.

The current content of a memory byte is lost whenever new information is placed in it.

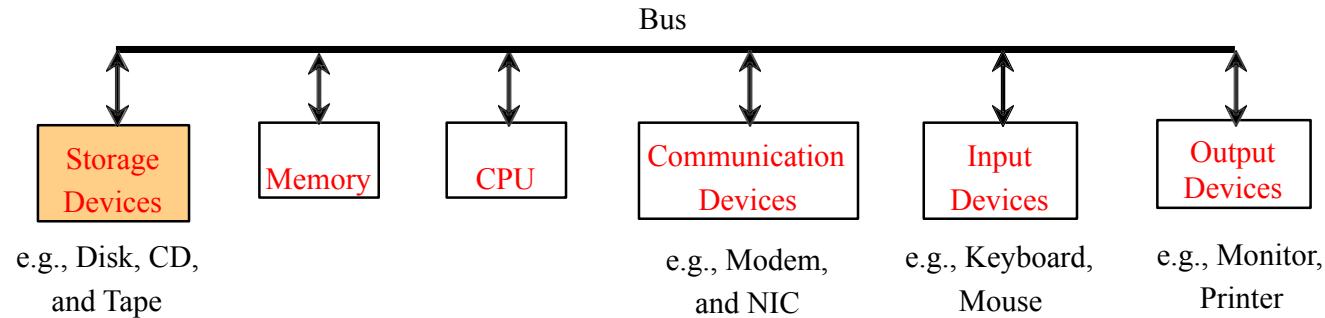


How Data is Stored?

- Data of various kinds, such as numbers, characters, and strings, are encoded as a series of bits (zeros and ones).
- Computers use zeros and ones because digital devices have two stable states, which are referred to as *zero* and *one* by convention.
- The programmers need not to be concerned about the encoding and decoding of data, which is performed automatically by the system based on the encoding scheme.
- For example, character 'J' is represented by 01001010 in one byte.
- A small number such as three can be stored in a single byte. If computer needs to store a large number that cannot fit into a single byte, it uses a number of adjacent bytes.

Memory address		Memory content	
.		.	
.		.	
.		.	
2000		01001010	Encoding for character 'J'
2001		01100001	Encoding for character 'a'
2002		01110110	Encoding for character 'v'
2003		01100001	Encoding for character 'a'
2004		00000011	Encoding for number 3

Storage Devices

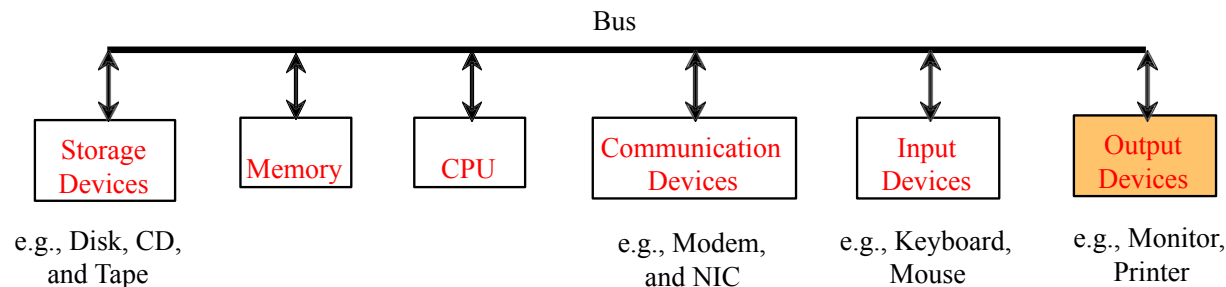


Memory is volatile, because information is lost when the power is off.

Programs and data are permanently stored on storage devices and are moved to memory when the computer actually uses them.

There are three main types of storage devices: Disk drives, Flash Drives, and Tape drives.

Input/Output Devices: Keyboard & Monitor



The monitor displays information (text and graphics).

The keyboard/mouse is used to input data to the computer.

What's a program?

- **Input** : Get data from some input device (keyboard, Mouse, etc.)
- **Output** : Display data on the screen or send to some other device.
- **Math & Logic** : Perform basic mathematical and logical operations.
- **Conditional execution** : Check for certain conditions, and execute the appropriate sequence of statements.
- **Repetition** : Perform some actions repeatedly.

That's all to it

Programs

- Computer *programs*, known as *software*, are instructions to the computer.
- You tell a computer what to do through programs. Without programs, a computer is an empty machine.
- Computers do not understand human languages, so you need to use computer languages to communicate with them.
- Programs are written using programming languages.

What's an Algorithm

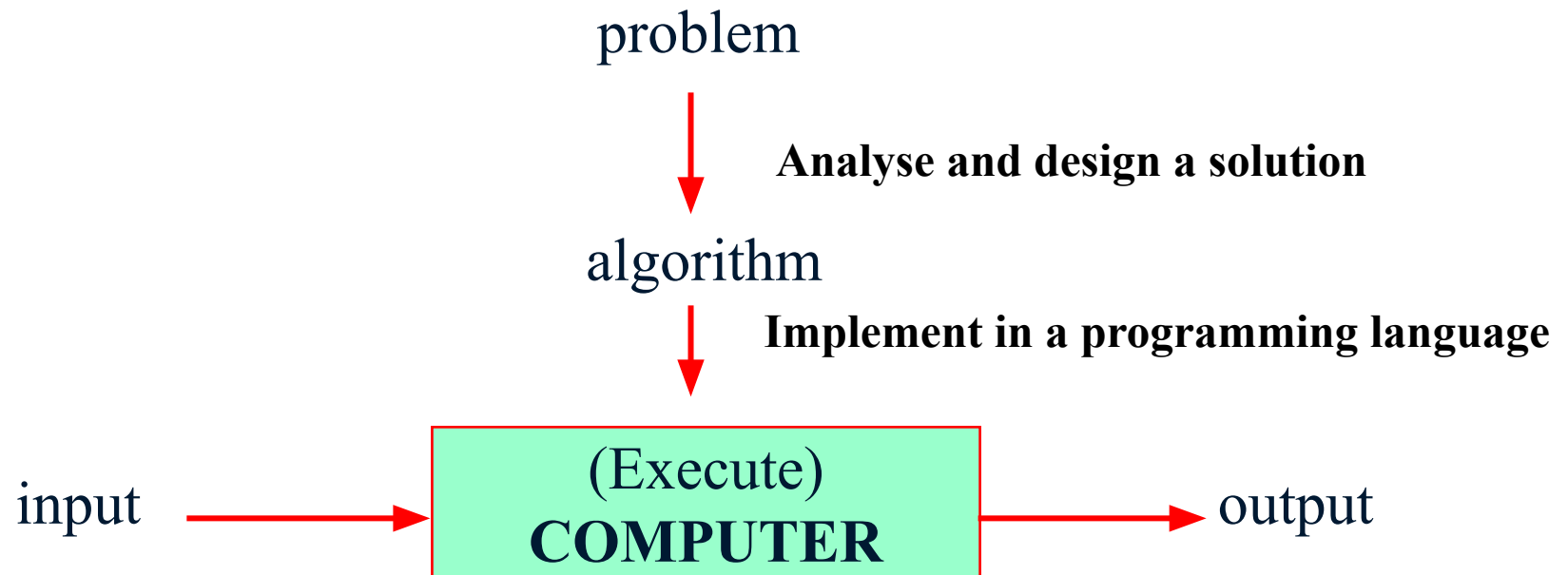
- If you want a computer to perform a task, you start by writing an algorithm.
- An algorithm is a sequence of **unambiguous** instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.
- It is like a recipe; a sequence (the order mattering) of actions to take to accomplish the given task, or a set of instructions written in a sequence that achieves a goal.
- The term comes from El Harezmi (Al Khowarizmi (many spellings possible...)) → “algorism” (originally) and then later “algorithm” come from his name.
- Developing algorithms is a fundamental problem solving skill.
- It has uses in many fields outside of Computer Science.

Algorithm: Formal Definition

- An algorithm describes a sequence of steps that is:
- Unambiguous
 - No “assumptions” are required to execute the algorithm.
 - The algorithm uses precise instructions.
- Executable
 - The algorithm should be carried out in practice.
- Terminating
 - The algorithm will eventually come to an end, or halt.

Problem Solving: Algorithm Design

- Algorithms are simply plans.
 - Detailed plans that describe the steps to solve a specific problem
- You already know quite a few
 - Calculate the area of a circle
 - Find the length of the hypotenuse of a triangle
- Some problems are more complex and require more steps
 - Calculate PI to 100 decimal places
 - Calculate the trajectory of a missile



A Simple Example

- A simple algorithm to get yourself a drink of orange juice.
- For simplicity, the following are true:
 - You have a clean glass in the cabinet
 - You have orange juice in your refrigerator
- So one valid algorithm might be:
 - get a glass from your cabinet.
 - go to the refrigerator and get the orange juice container
 - open the orange juice container
 - pour the orange juice from the container into the glass
 - put the orange juice container back in the refrigerator
 - drink your juice

Languages

- Natural Languages:
 - Languages people speak, such as English, Spanish, and French.
 - Not designed by people, evolved naturally.
- Formal Languages:
 - Languages designed by people for specific applications.
 - E.g., notations that mathematicians use is a formal language; good at denoting relationships among numbers and symbols.
- Computer Programs are written in programming languages,
 - which are formal languages that have been designed to Express computations.
 - They have strict rules about syntax, which come in two flavors:
 - Tokens (keywords), and structure.

Programming Languages

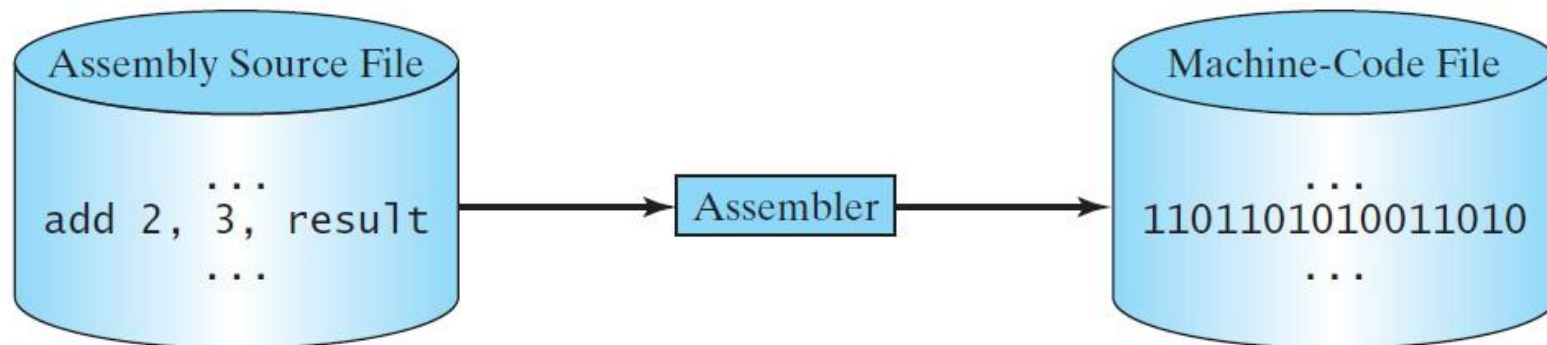
Machine Language Assembly Language High-Level Language

- Machine language is a set of primitive instructions built into every computer.
- The instructions are in the form of binary code, so you have to enter binary codes for various instructions.
- Program with native machine language is a tedious process.
- Moreover the programs are highly difficult to read and modify.
- For example, to add two numbers, you might write an instruction in binary like this:
 - 1101101010011010

Programming Languages

- Machine Language **Assembly Language** High-Level Language

- Assembly languages were developed to make programming easy.
- Since the computer cannot understand assembly language, however, a program called assembler is used to convert assembly language programs into machine code.
- For example, to add two numbers, you might write an instruction in assembly code like this:
 - ADD R1, R2, R3



Programming Languages

Machine Language Assembly Language High-Level Language

- The high-level languages are English-like and easy to learn and program.
- For example, the following is a high-level language statement that computes the area of a circle with radius 5, and stores the result in a variable (in memory):

```
area = 5 * 5 * 3.1415;
```

Popular High-Level Programming Languages

Language	Description
Ada	Named for Ada Lovelace, who worked on mechanical general-purpose computers. The Ada language was developed for the Department of Defense and is used mainly in defense projects.
BASIC	Beginner's All-purpose Symbolic Instruction Code. It was designed to be learned and used easily by beginners.
C	Developed at Bell Laboratories. C combines the power of an assembly language with the ease of use and portability of a high-level language.
C++	C++ is an object-oriented language, based on C.
C#	Pronounced "C Sharp." It is a hybrid of Java and C++ and was developed by Microsoft.
COBOL	COMmon Business Oriented Language. Used for business applications.
FORTRAN	FORmula TRANslation. Popular for scientific and mathematical applications.
Java	Developed by Sun Microsystems, now part of Oracle. It is widely used for developing platform-independent Internet applications.
Pascal	Named for Blaise Pascal, who pioneered calculating machines in the seventeenth century. It is a simple, structured, general-purpose language primarily for teaching programming.
Python	A simple general-purpose scripting language good for writing short programs.
Visual Basic	Visual Basic was developed by Microsoft and it enables the programmers to rapidly develop graphical user interfaces.

Pseudocode

- ***Pseudocode*** is an informal, language-independent way of describing an algorithm using a mixture of:
 - Natural language
 - Programming-like constructs
 - Mathematical notation
- ***Purpose:*** Focus on what the algorithm does and how it works, without worrying about programming-language syntax.
- ***Key characteristics***
 - Language-independent
 - Easy to read and understand
 - Expresses control flow and data manipulation
 - Not intended to be compiled or executed
 - Can be translated into any suitable programming language

Examples of Pseudocode

ALGORITHM FindMaximum(A)

$\text{max} \leftarrow A[0]$

 FOR each element x in A

 IF $x > \text{max}$ THEN

$\text{max} \leftarrow x$

 END IF

 END FOR

 RETURN max



ALGORITHM *SequentialSearch*($A[0..n - 1], K$)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

Advantages of Pseudocode

Think about the algorithm rather than syntax and language rules.



Language independent — Java, Python, C++, etc.



Easy to communicate algorithms to others



Identify logical errors before implementation



Analyze complexity without implementation details



Easy to modify during algorithm design

The same pseudocode can be implemented in:

Java

Python

C++



FindMaximum FindMaximum FindMaximum

The **algorithm remains the same**; only the implementation changes.

Important:

Pseudocode should be precise enough that another programmer can implement the algorithm, but abstract enough that programming-language syntax does not distract from the algorithm.

Why study algorithms?

- Theoretical importance
 - It is about the core of computer Science
- Practical importance
 - A practitioner's toolkit of known algorithms
 - Framework for designing and analyzing algorithms for new problems.
- When we construct an algorithm, we don't give an answer to a specific *instance* of a problem.
- Instead, we provide step by step instructions to solve **all** instances of the given problem.

Two main issues related to algorithms

- How to design algorithms
- How to analyze algorithms' efficiency
 - The theoretical study of computer program performance :
 - how fast it is?
 - Speed (time) —> complexity
 - how much memory it needs?
 - (Memory) space —> complexity
 - *Generally there is a tradeoff between space and time*
 - Other criteria:
 - Correctness, stability, simplicity, security, maintainability, modularity, robustness, functionality, usability —> architectural issues

Algorithm design techniques/strategies

- Brute force
- Divide and conquer
- Decrease and conquer
- Transform and conquer
- Greedy approach
- Dynamic programming
- Iterative improvement
- Backtracking
- Branch and bound

Important Problem Types

- sorting
- searching
- string processing
- graph problems
- combinatorial problems
- geometric problems
- numerical problems

Fundamental Data Structures

- list
 - array
 - linked list
 - string
- stack
- queue
- priority queue
- hash table
- graph
- tree
- set and dictionary

Fundamentals of Algorithmic Problem Solving

1. Understand the problem.
 - Do examples by hand.
 - Consider input range.
2. Choose between exact and approximate solutions.
3. Decide on appropriate data structures.
4. Design and specify the algorithm by using *Pseudocode*.
5. Show correctness of the algorithm.
6. Analyze your algorithm in terms of
 - efficiency/complexity (time and space),
 - simplicity,
 - generality
7. Implement your algorithms in a proper technology (programming language, operating system, etc.)
 - test
 - debug

First Algorithm Example: Searching

1. **Problem:** Given a sequence of items, determine whether a given item is present in the sequence, and if so, find its position (index).
2. We want an exact solution.
3. The appropriate data structure for this problem is **array**.
4. Here is the pseudocode for an algorithm to solve the problem:

ALGORITHM *SequentialSearch*($A[0..n - 1]$, K)

//Searches for a given value in a given array by sequential search

//Input: An array $A[0..n - 1]$ and a search key K

//Output: The index of the first element of A that matches K

// or -1 if there are no matching elements

$i \leftarrow 0$

while $i < n$ **and** $A[i] \neq K$ **do**

$i \leftarrow i + 1$

if $i < n$ **return** i

else return -1

First Algorithm Example: Searching

6. Analyze the algorithm.

- How many iterations of the while loop is needed for the algorithm to solve the problem and return (or stop)?

or

- How many comparison operations must be executed for the algorithm to solve the problem and return (or stop)?

- **Worst case** (the item is not present in the sequence or it is the last item in the sequence):
 - **$n-1$ iterations of the while loop or n comparisons**
- **Best case** (the item is the first item in the sequence)
 - **0 iterations or 2 comparisons**
- **Average case** (the item can be anywhere) $\approx n/2$
 - **depends on assumptions about the probabilistic distribution of input data, whether duplicates are allowed or not, etc.**

Second Algorithm Example: Binary Search

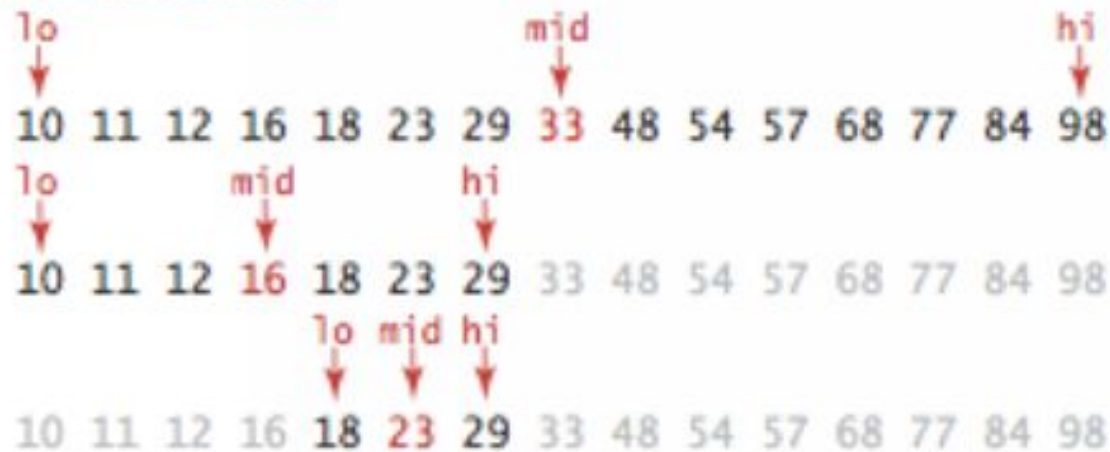
How the things can change if we already know that the input is already in sorted form (in descending or ascending order).

1. **Problem:** Given a sequence of items in sorted form (ascending), determine whether a given item is present in the sequence, and if so, find its position (index).
2. We want an exact solution.
3. The appropriate data structure for this problem is **array**.
4. Here is the pseudocode for an algorithm to solve the problem:

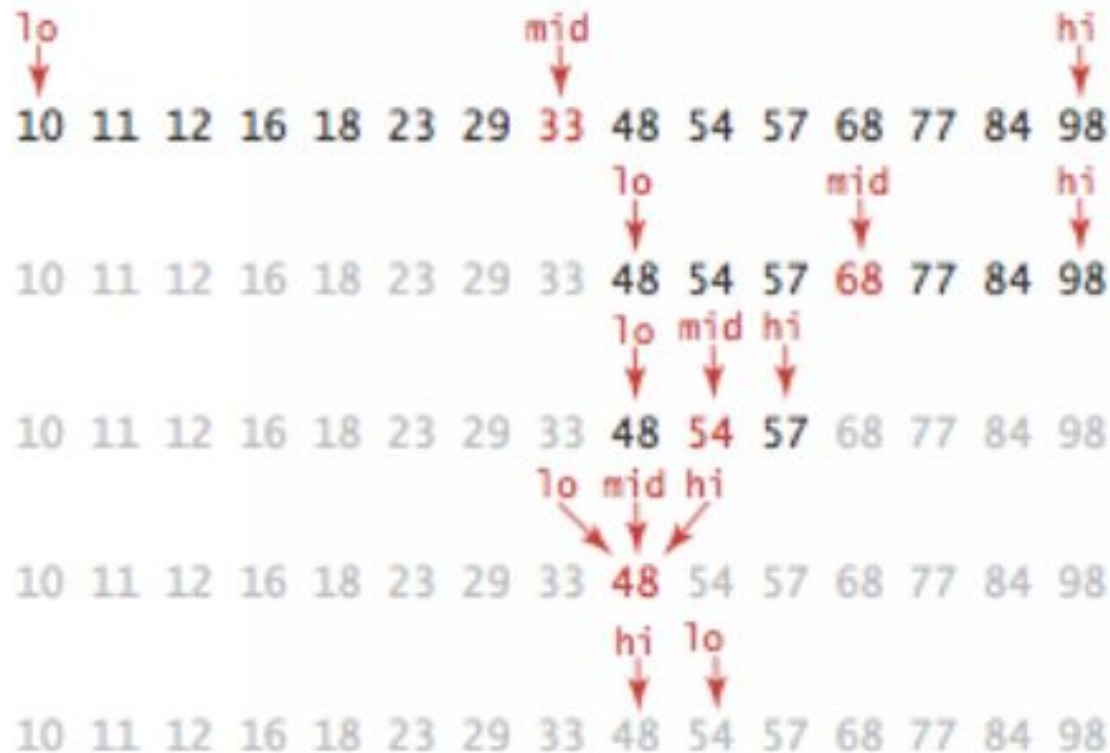
```
ALGORITHM BinarySearch( $A[0..n - 1]$ ,  $K$ )  
    //Implements nonrecursive binary search  
    //Input: An array  $A[0..n - 1]$  sorted in ascending order and  
    //      a search key  $K$   
    //Output: An index of the array's element that is equal to  $K$   
    //      or  $-1$  if there is no such element  
     $l \leftarrow 0$ ;  $r \leftarrow n - 1$   
    while  $l \leq r$  do  
         $m \leftarrow \lfloor (l + r) / 2 \rfloor$   
        if  $K = A[m]$  return  $m$   
        else if  $K < A[m]$   $r \leftarrow m - 1$   
        else  $l \leftarrow m + 1$   
    return  $-1$ 
```

Working principle of Binary Search

successful search for 23



unsuccessful search for 50



Second Algorithm Example: Binary Search

6. Analyze the algorithm.

- How many iterations of the while loop is needed for the algorithm to solve the problem and return (or stop)?

or

- How many times the search key is compared with an element of the array (**if** comparisons)?

- **Worst case** (the item is not present in the sequence or it is only possible to find it during the last iteration, e.g., the element is the minimum or maximum):
 - **$\log n$ iterations of $2 \cdot \log n$ comparisons**
- **Best case** (the item is sitting just in the middle of the sequence)
 - **0 iterations or 1 comparisons.**
- **Average case** (the item can be anywhere) $\approx$ **$\log n$ iterations**
 - **depends on assumptions about the probabilistic distribution of input data, whether duplicates are allowed or not, etc.**

Interesting facts about binary search

- The first binary search algorithm was designed in 1946.
- It was not possible to design and write an error free binary search algorithm until 1962.

The sorting problem

Ex. Student records in a university.

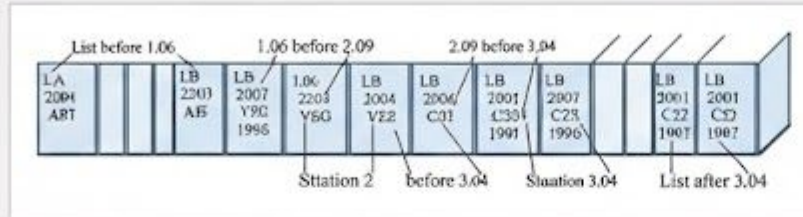
	Name	ID	Gender	Phone	Department
	Chen	3	A	614-470-0023	208 Math
	Rahde	2	A	212-741-8555	242 Physics
	Gazel	4	B	780-093-9872	101 Chemistry
item →	Fafla	1	A	760-693-5675	101 Biology
	Kanaga	3	B	604-112-8640	221 Biology
	Andrews	1	A	564-380-0022	607 Literature
key →	Baide	4	C	874-628-1212	121 Mathman

Sort. Rearrange array of N items into ascending order.

Name	ID	Gender	Phone	Department
Andrews	1	A	564-380-0022	607 Literature
Baide	4	C	874-628-1212	121 Mathman
Chen	3	A	614-470-0023	208 Math
Fafla	1	A	760-693-5675	101 Biology
Gazel	4	B	780-093-9872	101 Chemistry
Kanaga	3	B	604-112-8640	221 Biology
Rahde	2	A	212-741-8555	242 Physics

Sorting applications

Sorting applications



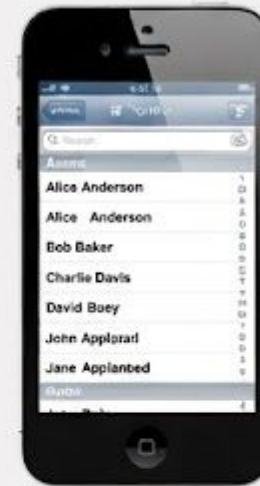
Library of Congress numbers



FedEx packages



playing cards



contacts



Hogwarts houses

Third Algorithm Example: Selection Sort

Same problem, same data structure, exact solution expected.
Here is the pseudocode for an algorithm to solve the problem via another sorting algorithm:

ALGORITHM *SelectionSort*($A[0..n - 1]$)

//Sorts a given array by selection sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

$min \leftarrow i$

for $j \leftarrow i + 1$ **to** $n - 1$ **do**

if $A[j] < A[min]$ $min \leftarrow j$

 swap $A[i]$ and $A[min]$

Working principle of Selection Sort

We want to sort the following sequence of numbers: 89, 45, 68, 90, 29, 34, 17

First iteration of outer loop		89	45	68	90	29	34	17
First iteration of outer loop	17		45	68	90	29	34	89
First iteration of outer loop	17	29		68	90	45	34	89
First iteration of outer loop	17	29	34		90	45	68	89
First iteration of outer loop	17	29	34	45		90	68	89
First iteration of outer loop	17	29	34	45	68		90	89
Result array when algorithm stops	17	29	34	45	68	89		90

Each line corresponds to one iteration of the algorithm, i.e., a pass through the list's tail to the right of the vertical bar; an element in bold indicates the smallest element found. Elements to the left of the vertical bar are in their final positions and are not considered in this and subsequent iterations.

Third Algorithm Example: Selection Sort

- **Worst case** (the given sequence of items are sorted in reverse order):
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Best case** (the given sequence of items are already sorted in intended order)
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Average case** (the given sequence of items are arbitrarily ordered)
 - n^2 iterations of the inner for loop or n^2 comparisons.

Thus, selection sort is a n^2 algorithm on all inputs

Fourth Algorithm Example: Bubble Sort

1. **Problem:** Given a sequence of items, rearrange them in-place in an increasing (ascending) sorted fashion.
2. **We want an exact solution.**
3. The appropriate data structure for this problem is **array**.
4. Here is the pseudocode for an algorithm to solve the problem:

ALGORITHM *BubbleSort*($A[0..n - 1]$)

//Sorts a given array by bubble sort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n - 2$ **do**

for $j \leftarrow 0$ **to** $n - 2 - i$ **do**

if $A[j + 1] < A[j]$ **swap** $A[j]$ and $A[j + 1]$

Working principle of Bubble Sort

We want to sort the following sequence of numbers: **89, 45, 68, 90, 29, 34, 17**

First iteration of outer loop	89	$\overset{?}{\leftrightarrow}$	45		68		90		29		34		17
	45		89	$\overset{?}{\leftrightarrow}$	68		90		29		34		17
	45		68		89	$\overset{?}{\leftrightarrow}$	90	$\overset{?}{\leftrightarrow}$	29		34		17
	45		68		89		29		90	$\overset{?}{\leftrightarrow}$	34		17
	45		68		89		29		34		90	$\overset{?}{\leftrightarrow}$	17
	45		68		89		29		34		17		90
Second iteration of outer loop	45	$\overset{?}{\leftrightarrow}$	68	$\overset{?}{\leftrightarrow}$	89	$\overset{?}{\leftrightarrow}$	29		34		17		90
	45		68		29		89	$\overset{?}{\leftrightarrow}$	34		17		90
	45		68		29		34		89	$\overset{?}{\leftrightarrow}$	17		90
	45		68		29		34		17		89		90
Third iteration	etc.												

A new line is shown after a swap of two elements is done. The elements to the right of the vertical bar are in their final positions and are not considered in subsequent iterations of the algorithm.

Fourth Algorithm Example: Bubble Sort

6. Analyze the algorithm.

- How many iterations of the inner for loop is needed for the algorithm to solve the problem and return (or stop)?

or

- How many comparison operations (**if** statements) must be executed for the algorithm to solve the problem and return (or stop)?

- **Worst case** (the given sequence of items are sorted in reverse order):
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Best case** (the given sequence of items are already sorted in intended order)
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Average case** (the given sequence of items are arbitrarily ordered)
 - n^2 iterations of the inner for loop or n^2 comparisons.

Bubble Sort: Improved Version

ALGORITHM *BubbleSort*($A[0..n-1]$)

// Sorts a given sequence of values by bubble sort.

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

for $i \leftarrow 0$ **to** $n-2$ **do**

$swapped \leftarrow false$

for $j \leftarrow 0$ **to** $n-2-i$ **do**

if ($A[j+1] < A[j]$) **then**

 swap $A[j]$ and $A[j+1]$

$swapped \leftarrow true$

#if no item was swapped that means sorted.

if not($swapped$) **return**

- **Worst case :**
 - n^2 iterations of the inner for loop or n^2 comparisons
- **Best case**
 - n iterations of the inner for loop or n comparisons
- **Average case** (depends ...) $\approx n^2$ iterations or comparisons

Sorting Algorithm Visualizer

A nice web site that helps understand how several sorting algorithms work

Have a look!

[Interactive Sorting Algorithm Visualiser](#)